

TRABAJO PRÁCTICO Nº2

Clases - Herencia

EJERCICIO 1

Declarar una clase Funcionario que herede de la clase Persona (nombre, apellido, dni), con un atributo que indique el poder al que pertenece (puede ser ejecutivo, legislativo o judicial).

Declarar la clase legislador, que herede de la clase funcionario y contenga un atributo que indique a qué provincia representa.

Declarar la clase diputado y senador como sub clases de Legislador.

Declarar la clase Juez. Los jueces deben tener un atributo que indique su jurisdicción (provincial o nacional), y al igual que los legisladores un atributo que indique a qué poder pertenece.

Todas las clases deben tener un método toString() que indique el estado del objeto (valores de sus atributos en un momento específico).

EJERCICIO 2

Crear una clase Persona, que contenga el atributo: nombre, altura.

Crear la clase Superhéroe que herede de Persona

Cada superhéroe debe tener sus atributos: nombFicticio, tipoPoder.

Se debe mostrar por pantalla el nombre real, el nombre ficticio, altura y tipo de poder

EJERCICIO 3

Crear una clase **Pokémon** donde puedas guardar su nombre, color, potencia de ataque y nivel de vida (0-100) así como los constructores (1) por defecto y (2) con parámetros específicos, nombre y color, cuando no es por defecto, su salud es de 100 y su poder es de 50.

Añade las funciones get y set para cada atributo y una más que permita al Pokémon saludar con su nombre. "Hola soy {nombre_pokemon}"

Crear 1 método que permita sanar al Pokémon y otro que le permita evolucionar cambiando de nombre, color y aumentando un 20% su potencia de ataque

EJERCICIO 4

Definir una clase Libro para manejar la información asociada a un libro. La información de interés para un libro es: el título, el autor y el precio. Los métodos de interés son:

- Un constructor para crear un objeto libro, con título y autor como parámetros.
- Imprimir en pantalla el título, los autores y el precio del libro.
- Métodos get y set para cada atributo de un libro.

Se debe extender la clase Libro definiendo las siguientes clases:

- Libros físicos con un nuevo atributo que especifica si es tapa dura o no.
- Novelas: pueden ser de diferente tipo: histórica, romántica, policíaca, realista, ciencia ficción, aventuras, etc.

Para cada una de las clases anteriores se debe definir su constructor y redefinir adecuadamente el método para visualizar el objeto (toString)

EJERCICIO 5

Crear una clase Animal que muestre el nombre, la edad y raza y sus clases hijas (Gato, Pajaro, Perro).

La clase Animal tendrá un método a través del cual cada clase hija deberá mostrar luego un mensaje por pantalla informando de que se alimenta.

Implementar los constructores, getters y setters correspondientes.

Crear una clase Principal, instanciar un objeto de cada tipo de animal e invocar el método que indique de qué se alimenta cada animal.

Crear diagrama UML.

EJERCICIO 6

Crear una clase llamada Electrodoméstico que contenga los siguientes datos: precio, color, consumo energético (debe ser una letra entre la A y la F) y marca.

- Al crear un objeto de este tipo se deben recibir todos sus atributos por parámetros.
- Se debe validar que la letra ingresada en el constructor sea correcta (indistintamente si es minúscula o mayúscula). En caso de ser incorrecta debe asignarse F por defecto.

Crear las clases Lavadora y Televisor, que hereden de Electrodoméstico.

- Lavadora debe guardar los atributos revoluciones y cargaMax
- El televisor debe guardar las pulgadas.
- Al crear un nuevo objeto Televisor o Lavadora serán obligatorios todos los atributos de la superclase como de la subclase.

Crear clase Principal, instanciar un objeto televisor y otro lavadora y llamar al método que permite imprimir por pantalla su estado.

EJERCICIO 7

Se requiere crear un sistema para una concesionaria. La misma vende motos y autos, de los cuales se guardará marca, modelo (año de creación), patente, precio y Motor.

Además todos tendrán los métodos: encender, acelerar y frenar.

- Cada motor cuenta con cilindrada en centímetros cúbicos, número de motor y kilometraje.
- Los Autos tendrán un método para encender y apagar el aire acondicionado: boolean
- Las motos tendrán una cilindrada y un método esGamaAlta() que devuelva true cuando su cilindrada es mayor a 500cc o false cuando no lo es.

Crear la clase Principal, que cree un objeto Moto y otro Auto, ejecute los métodos encenderAireAcondicionado(), esGamaAlta() y métodos para mostrar el estado del auto. Crear diagrama UML.

EJERCICIOS ENTREGABLES:

Los ejercicios 8, 9 y 10 son los que se deben entregar.

El método de entrega es compartiendo un repositorio en github, con la siguiente estructura:

```
TP2 - HERENCIA
|_ EjercicioEntregable8
    |_ Clases correspondientes .java
    |_ Principal.java
|_ EjercicioEntregable9
    |_ Clases correspondientes .java
    |_ Principal.java
|_ EjercicioEntregable10
    |_ Clases correspondientes .java
    |_ Principal.java
|_ EjercicioEntregableN
    |_ Clases correspondientes .java
    |_ Principal.java
```

EJERCICIO 8

Se requiere desarrollar un sistema para una terminal de cobro que procese pago con tarjetas de crédito.

Las tarjetas de crédito guardan el nombre de la entidad financiera que las emiten (Visa, Mastercard, American Express}, el nombre de la entidad bancaria a la que pertenecen, número de tarjeta (xxxx-xxxx-xxxx-xxxx), el saldo disponible y los datos del titular: DNI, nombre, apellido, teléfono y mail.

Todos estos datos serán necesarios a la hora de crear una nueva tarjeta.

A la hora de abonar un pago, la terminal de cobro recibirá la tarjeta con la que desea hacerse el pago, junto con el monto que se desea abonar y la cantidad de cuotas (de 1 a 6).

- Si el pago es en 1 cuota, no se genera ningún recargo, de lo contrario, el monto se incrementará en un 3% por cada cuota superior a 1. (Ejemplo: Pagar en 4 cuotas representará un 9% de incremento).

El POSNET debe chequear que la tarjeta tenga saldo suficiente para poder efectuar el pago (incluido el recargo, si existiese). Si la operación es exitosa, la terminal de cobro debe retornar un ticket donde consten los siguientes datos: Nombre y apellido del cliente, monto total a pagar, monto de cada cuota. Si la operación no tuvo éxito, se retornará null.

Puntos a desarrollar

1. Desarrollar el diagrama de clases UML que modele lo enunciado y donde consten las clases con sus atributos, métodos y relaciones (los constructores pueden omitirse).
2. Crear un proyecto en Java que resuelva:
 - a. Desarrollar, en la clase Posnet, el método efectuarPago(), cuyos parámetros, lógica y valor de retorno deben deducirse según lo enunciado. Desarrollar también los métodos derivados que puedan surgir de él para conseguir el objetivo.
 - b. Desarrollar el método main del proyecto y generar las instancias necesarias para poder efectuar un pago de \$10000 en 5 cuotas, usando una tarjeta de crédito con saldo disponible de \$15000 (el resto de los datos, pueden inventarse a gusto).

EJERCICIO 9

Un comerciante de nuestra ciudad necesita un sistema que le permita agilizar los cálculos al momento de pagar los sueldos de sus empleados.

El comercio cuenta con vendedores, cajeros, administrativos y repartidores. Todos guardan nombre, dni, domicilio y fecha de ingreso (año).

La totalidad de los empleados cobran un sueldo básico de \$650000.

- Los vendedores deben guardar el total de las ventas mensuales. A su sueldo se le debe sumar un 0.5% de comisión sobre lo que vendieron.
- Los administrativos tienen la posibilidad de elegir su jornada laboral (jornada completa o media jornada) Deducir su sueldo a partir de las horas que trabaja.
- Los repartidores cobran 500 pesos extra por cada reparto que realicen. Se debe guardar un atributo que contabilice los repartos mensuales.

Puntos a desarrollar:

- Diagrama UML
- Implementar la clase Ejercicio2, instanciar 1 empleado de cada tipo y llamar metodo imprimirRecibo() que muestre por pantalla los datos del empleado y su sueldo a cobrar en detalle.

Ejemplo:

Nombre: Juan Perez

Dni: [xx.xxx.xxx](#)

Domicilio: Alem 1111

Fecha de ingreso: 2020

Categoría: Administrativo

Sueldo básico: \$650000

Comisiones por ventas: \$50000

Total a cobrar: \$700000

EJERCICIO 10

Se requiere realizar un programa que modele diferentes tipos de inmuebles para una inmobiliaria.

Cada **inmueble** tiene los siguientes atributos:

- *identificador inmobiliario numérico de 5 dígitos*
- *área en metros cuadrados*
- *dirección*

Debe contar con un método imprimir() que muestre por pantalla todos sus atributos.

Los **inmuebles** pueden ser **para uso de vivienda** o para uso comercial. Las inmuebles viviendas pueden ser **casas o apartamentos**, los cuales tienen los siguientes atributos:

- *número de habitaciones*
- *número de baños.*

Las **casas** pueden estar en un **conjunto cerrado** o **ser independientes**. A su vez, las **casas en conjunto cerrado** tienen como atributo el *valor de la administración y si incluyen o no áreas comunes* como piscinas y campos deportivos.

De otro lado, los **apartamentos** pueden ser **monoambiente** o **departamentos familiares**.

Los **apartamentos** pagan un *valor de administración*, los **monoambientes** no tienen *habitación* y los departamentos familiares mínimo 1 habitación.

Los **inmuebles de uso comercial** se clasifican en **locales comerciales** y **oficinas**.

Los **locales comerciales** tienen como atributo su *localización* (si es interno o da a la calle).

Los **locales comerciales** tienen un atributo para conocer el *centro comercial* donde están establecidos.

Cada inmueble tiene un valor de m2.

- **Casa rural:** 1 500 000
- **Casa en conjunto privado:** 2 500 000
- **Casa independiente:** 2 000 000
- **Monoambiente:** 1 000 000
- **Departamento familiar:** 1 800 000
- **Local comercial:** 3 000 000
- **Oficina:** 2 400 000

Todas las clases deben tener el método `imprimir()` que a su vez invoque al método de la clase a la que extienden.

Crear una clase Principal, y con su método `main` que cree dos inmuebles, calcule su precio de venta de acuerdo al área y se muestren sus datos en pantalla.